

Collection summary

1/ ArrayList

```
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {
        ArrayList<Integer> arrayList = new ArrayList<>();

        // 1. عملية الإضافة (Add)
        // أ (الإضافة في النهاية)
        arrayList.add(10);
        arrayList.add(20);

        // ب (الإضافة في البداية) في المؤشر 0
        arrayList.add(0, 5);

        // ج (الإضافة في المنتصف)
        int middleIndex = arrayList.size() / 2;
        arrayList.add(middleIndex, 15);

        // شكل القائمة الآن: 5, 10, 15, 20
        // 2. عملية التحديث (Update)
        // تحديث العنصر في المؤشر 2 ليصبح 99
        arrayList.set(2, 99);

        // 3. عملية التحديد/الاسترجاع (Select/Get)
        // استرجاع العنصر الموجود في المؤشر 1
        int selectedValue = arrayList.get(1);
        System.out.println("القيمة المحددة: " + selectedValue);

        // 4. عملية الحذف (Remove)
        // حذف العنصر الموجود في المؤشر 3
        arrayList.remove(3);
    }
}
```

.....

2/LinkedList

```
import java.util.LinkedList;

public class LinkedListExample {
    public static void main(String[] args) {
        LinkedList<Integer> linkedList = new LinkedList<>();

        // 1. عملية الإضافة (Add)
        // أ) الإضافة في النهاية
        linkedList.addLast(10); // أيضاً add(10) يمكن استخدام
        linkedList.addLast(20);

        // ب) الإضافة في البداية
        linkedList.addFirst(5); // أيضاً add(0, 5) يمكن استخدام

        // ج) الإضافة في المنتصف
        int middleIndex = linkedList.size() / 2;
        linkedList.add(middleIndex, 15);

        // شكل القائمة الآن: 5, 10, 15, 20
        // 2. عملية التحديث (Update)
        // تحديث العنصر في المؤشر 2 ليصبح 99
        linkedList.set(2, 99);

        // 3. عملية التحديد/الاسترجاع (Select/Get)
        // استرجاع العنصر الموجود في المؤشر 1
        int selectedValue = linkedList.get(1);
        // لاسترجاع أول أو آخر عنصر مباشرة: linkedList.getFirst() / linkedList.getLast()
        System.out.println("القيمة المحددة: " + selectedValue);

        // 4. عملية الحذف (Remove)
        // حذف العنصر في المؤشر 0
        linkedList.remove(0);
        // linkedList.removeFirst() أو linkedList.removeLast() يمكن أيضاً استخدام
    }
}
```

3/vector

```
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {
```

```

Vector<Integer> vector = new Vector<>();

// 1. عملية الإضافة (Add)
// أ (الإضافة في النهاية)
vector.add(10);
vector.add(20);

// ب (الإضافة في البداية) في المؤشر 0
vector.add(0, 5); // يمكن أيضاً استخدام vector.insertElementAt(5, 0);

// ج (الإضافة في المنتصف)
int middleIndex = vector.size() / 2;
vector.add(middleIndex, 15);

// شكل المتجه الآن: [5, 10, 15, 20]

// 2. عملية التحديث (Update)
// تحديث العنصر في المؤشر 2 ليصبح 99
vector.set(2, 99);

// 3. عملية التحديد/الاسترجاع (Select/Get)
// استرجاع العنصر الموجود في المؤشر 1
int selectedValue = vector.get(1);
System.out.println("القيمة المحددة: " + selectedValue);

// 4. عملية الحذف (Remove)
// حذف العنصر في المؤشر 3
vector.remove(3);
}
}

```

.....

4/Stack

```

import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();

        // 1. الإضافة (Push) - دائماً في القمة (النهاية)
        stack.push(10);
        stack.push(20);
        stack.push(30);

        // 2. التحديث (Update) - لا يدعم التحديث المباشر برمجياً إلا كونه
        // لتحديث العنصر في المنتصف (رغم أنه غير محبذ في المكدسات) index نستخدم
        stack.set(1, 99);
    }
}

```

```
// رؤية العنصر الموجود في القمة دون حذفه - (Select/Peek) الاسترجاع 3.
int topElement = stack.peek(); // سيعيد 30

// إزالة العنصر الموجود في القمة - (Pop) الحذف 4.
int removedElement = stack.pop(); // سيحذف ويعيد 30
}
}
```

5/ PriorityQueue

```
import java.util.PriorityQueue;

public class PriorityQueueExample {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();

        // 1. الإضافة (Add/Offer) - الهيكل يرتبها تلقائياً
        pq.offer(30);
        pq.offer(10); // سيكون هو الأول لأنه الأصغر
        pq.offer(20);

        // 2. التحديث (Update) - يجب حذف القديم وإضافة الجديد
        pq.remove(20);
        pq.offer(25);

        // 3. رؤية العنصر ذو الأولوية القصوى (الأصغر) - (Select/Peek) الاسترجاع
        int highestPriority = pq.peek(); // سيعيد 10

        // 4. إزالة العنصر ذو الأولوية القصوى - (Remove/Poll) الحذف
        pq.poll(); // سيحذف 10
    }
}
```

6/ArrayDeque

```
import java.util.ArrayDeque;

public class ArrayDequeExample {
    public static void main(String[] args) {
        ArrayDeque<Integer> deque = new ArrayDeque<>();

        // 1. الإضافة (Add)
        deque.addLast(20); // في النهاية
        deque.addLast(30);
        deque.addFirst(10); // في البداية

        // 2. التحديث (Update) - لا يدعم التحديث المباشر للمؤشرات
        // 3. الاسترجاع (Select) - من البداية أو النهاية
        int first = deque.getFirst(); // 10
        int last = deque.getLast();   // 30

        // 4. الحذف (Remove)
        deque.removeFirst(); // 10 يحذف
        deque.removeLast();  // 30 يحذف
    }
}
```

.....

7/HashSet

```
import java.util.HashSet;
import java.util.LinkedHashSet;

public class SetExamples {
    public static void main(String[] args) {
        // لتحافظ على ترتيب الإضافة LinkedHashSet بـ HashSet يمكن تبديل
        HashSet<String> set = new HashSet<>();

        // 1. الإضافة (Add)
        set.add("Apple");
        set.add("Banana");
        set.add("Orange");

        // 2. التحديث (Update) - لا يوجد تحديث مباشر في الـ Sets.
        // يجب حذف العنصر القديم وإضافة الجديد
    }
}
```

```

        if (set.contains("Banana")) {
            set.remove("Banana");
            set.add("Mango");
        }

        // 3. الاسترجاع (Select) - نتحقق فقط من وجود العنصر أو نمر على جميع العناصر
        boolean hasApple = set.contains("Apple");

        // 4. الحذف (Remove)
        set.remove("Orange");
    }
}

```

.....

8/LinkedHashSet

```

import java.util.LinkedHashSet;
import java.util.Iterator;

public class LinkedHashSetExample {
    public static void main(String[] args) {
        LinkedHashSet<String> linkedSet = new LinkedHashSet<>();

        // =====
        // 1. عملية الإضافة (Add)
        // =====
        // الإضافة الافتراضية (تضاف العناصر بالترتيب في النهاية)
        linkedSet.add("Ahmed");
        linkedSet.add("Sara");
        linkedSet.add("Omar");

        /*
        * ✨ (وما فوق Java 21 فقط في) ميزة جديدة:
        * يمكنك الآن الإضافة في البداية والنهاية بشكل صريح:
        * linkedSet.addFirst("Ali");
        * linkedSet.addLast("Ziad");
        */

        // (Index) ملاحظة: لا يمكن الإضافة في "المنتصف" لأنه لا يوجد مؤشر

        // =====
        // 2. عملية التحديث (Update)
    }
}

```

```
// =====
// لا يوجد تحديث مباشر. يجب حذف العنصر القديم وإضافة الجديد.
// ⚠ تحذير: عند فعل ذلك، العنصر الجديد سيتم إضافته في "نهاية" المجموعة ولن يأخذ مكان القديم
if (linkedSet.contains("Sara")) {
    linkedSet.remove("Sara");
    linkedSet.add("Salma"); // "Salma" ستصبح في النهاية
}

// =====
// 3. عملية التحديد/الاسترجاع (Select)
// =====
// أ (التحقق من الوجود) سريع جداً
boolean hasOmar = linkedSet.contains("Omar");

// (للإصدارات القديمة من جافا Iterator باستخدام) ب (استرجاع أول عنصر)
Iterator<String> iterator = linkedSet.iterator();
if (iterator.hasNext()) {
    String firstElement = iterator.next(); // سيعيد "Ahmed"
    System.out.println("أول عنصر: " + firstElement);
}

/*
 * ✨ ميزة جديدة (وما فوق Java 21 فقط في):
 * String first = linkedSet.getFirst();
 * String last = linkedSet.getLast();
 */

// =====
// 4. عملية الحذف (Remove)
// =====
// الحذف يتم عبر تمرير القيمة نفسها وليس المؤشر
linkedSet.remove("Omar");

/*
 * ✨ ميزة جديدة (وما فوق Java 21 فقط في):
 * linkedSet.removeFirst();
 * linkedSet.removeLast();
 */
}
}
```

.....

9/HashMap

```
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Hashtable;

public class MapExamples {
    public static void main(String[] args) {
        // LinkedHashMap كمثال، ويمكنك استبدالها بـ HashMap استخدمنا
        HashMap<Integer, String> map = new HashMap<>();

        // 1. الإضافة (Put) - تعتمد على المفتاح (Key) وليس البداية أو النهاية
        map.put(1, "Ahmed");
        map.put(2, "Sara");
        map.put(3, "Omar");

        // 2. التحديث (Update/Replace)
        // لنفس المفتاح، سيتم التحديث التلقائي put إذا استخدمت
        // لتكون العملية أوضح replace أو يمكنك استخدام:
        map.put(2, "Ali");
        map.replace(3, "Omar", "Ziad");

        // 3. الاستخدام المفتاح - (Select/Get) الاسترجاع
        String selectedName = map.get(1); // سيعيد "Ahmed"

        // 4. الحذف (Remove) - باستخدام المفتاح
        map.remove(2); // سيحذف (2, "Ali")
    }
}
```

10/HashTable

```
import java.util.Hashtable;

public class HashtableExample {
    public static void main(String[] args) {
        Hashtable<String, String> hashtable = new Hashtable<>();

        // =====//
        // 1. الإضافة (Add)
        // =====//
        // تضاف العناصر بناءً على خوارزمية التجزئة، الترتيب هنا عشوائي تماماً
        hashtable.put("EG", "Egypt")
    }
}
```



```

;hashtable.put("SA", "Saudi Arabia")
;hashtable.put("AE", "UAE")

// محاولة إضافة null ستؤدي إلى خطأ NullPointerException
// خطأ ✖ ;hashtable.put("US", null) //

===== //
2. التحديث (Update)
===== //
// التحديث عبر المفتاح فقط
;hashtable.put("EG", "Arab Republic of Egypt")

===== //
3. الاسترجاع / التحديث (Select)
===== //
// الاسترجاع عبر المفتاح
;String country = hashtable.get("SA")

// لا توجد طريقة مباشرة لاسترجاع "أول" أو "آخر" عنصر لأن المفهوم غير موجود أصلاً.

===== //
4. الحذف (Remove)
===== //
// الحذف عبر المفتاح
;hashtable.remove("AE")
{
{

```

11/LinkedHashMap

```

;import java.util.LinkedHashMap
;import java.util.Map

} public class LinkedHashMapExample
} public static void main(String[] args)
;()<>LinkedHashMap<String, Integer> linkedMap = new LinkedHashMap

===== //
1. الإضافة (Add)
===== //
// الإضافة العادية (تتم دائماً في النهاية)
;linkedMap.put("Ali", 90)
;linkedMap.put("Sara", 85)

*/
* ❁ في Java 21 وما فوق:

```

```
* يمكنك الإضافة في البداية والنهاية بشكل صريح:
// يضاف في البداية * ;linkedMap.putFirst("Ahmed", 95)
// يضاف في النهاية * ;linkedMap.putLast("Omar", 80)
/*
```

```
//
===== //
2. التحديث (Update)
```

```
//
===== //
// يتم التحديث باستخدام المفتاح. (هذا لن يغير ترتيب العنصر، سيبقى في مكانه)
// تحديث درجة سارة ;linkedMap.put("Sara", 99)
// أو بشكل أكثر دقة:
;linkedMap.replace("Ali", 95)
```

```
//
===== //
3. الاسترجاع / التحديد (Select)
```

```
//
===== //
// الاسترجاع عبر المفتاح (بغض النظر عن مكانه)
;int saraScore = linkedMap.get("Sara")
```

```
*/
* ✨ في Java 21 وما فوق لمعرفة البداية والنهاية:
;()Map.Entry<String, Integer> first = linkedMap.firstEntry *
;()Map.Entry<String, Integer> last = linkedMap.lastEntry *
;((first.getKey() + " أول عنصر: ")System.out.println *
/*
```

```
//
===== //
4. الحذف (Remove)
```

```
//
===== //
// الحذف يتم عبر المفتاح
;linkedMap.remove("Ali")
```

```
*/
* ✨ في Java 21 وما فوق للحذف من البداية والنهاية:
;()linkedMap.pollFirstEntry * // يحذف ويرجع أول عنصر
;()linkedMap.pollLastEntry * // يحذف ويرجع آخر عنصر
/*
{
{
```

.....